# **Self-supervised Learning**
# Generative Models

Hermann Blum

UNIVERSITÄT BONN

LAMARR INSTITUTE FOR MACHINE LEARNING AND ARTIFICIAL INTELLIGENCE

Robot Perception and Learning Lab
rpl.uni-bonn.de

# Modelling of p(X)

We have some data samples

We know that they all come from the same distribution

We fit an (approximate) model q(X)

We can sample new points from q(X)

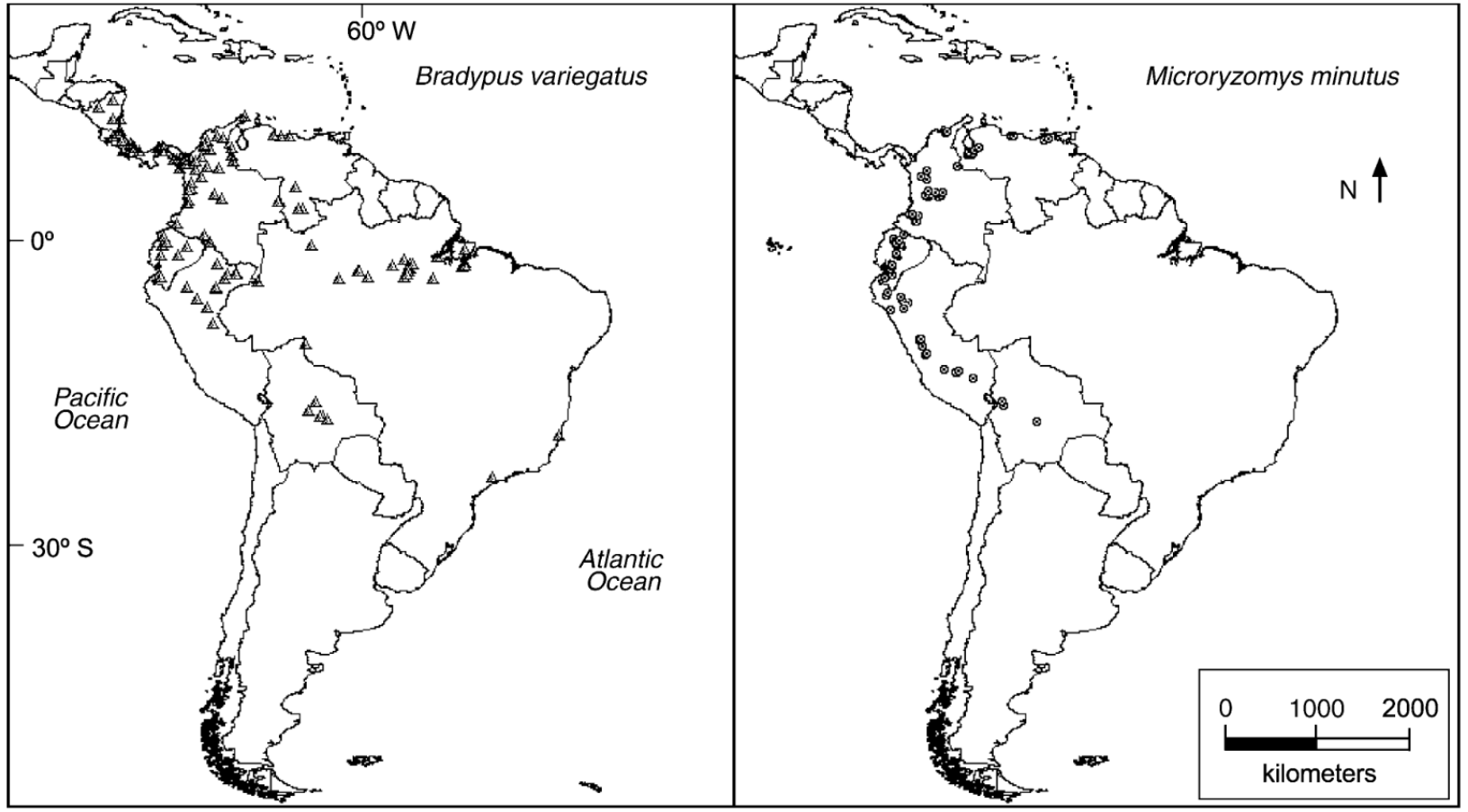
# Why is this self-supervised?

**Data:** Only raw data (X) is required, no labels at all

**Learning Outcome:** Learns hidden structure in the data

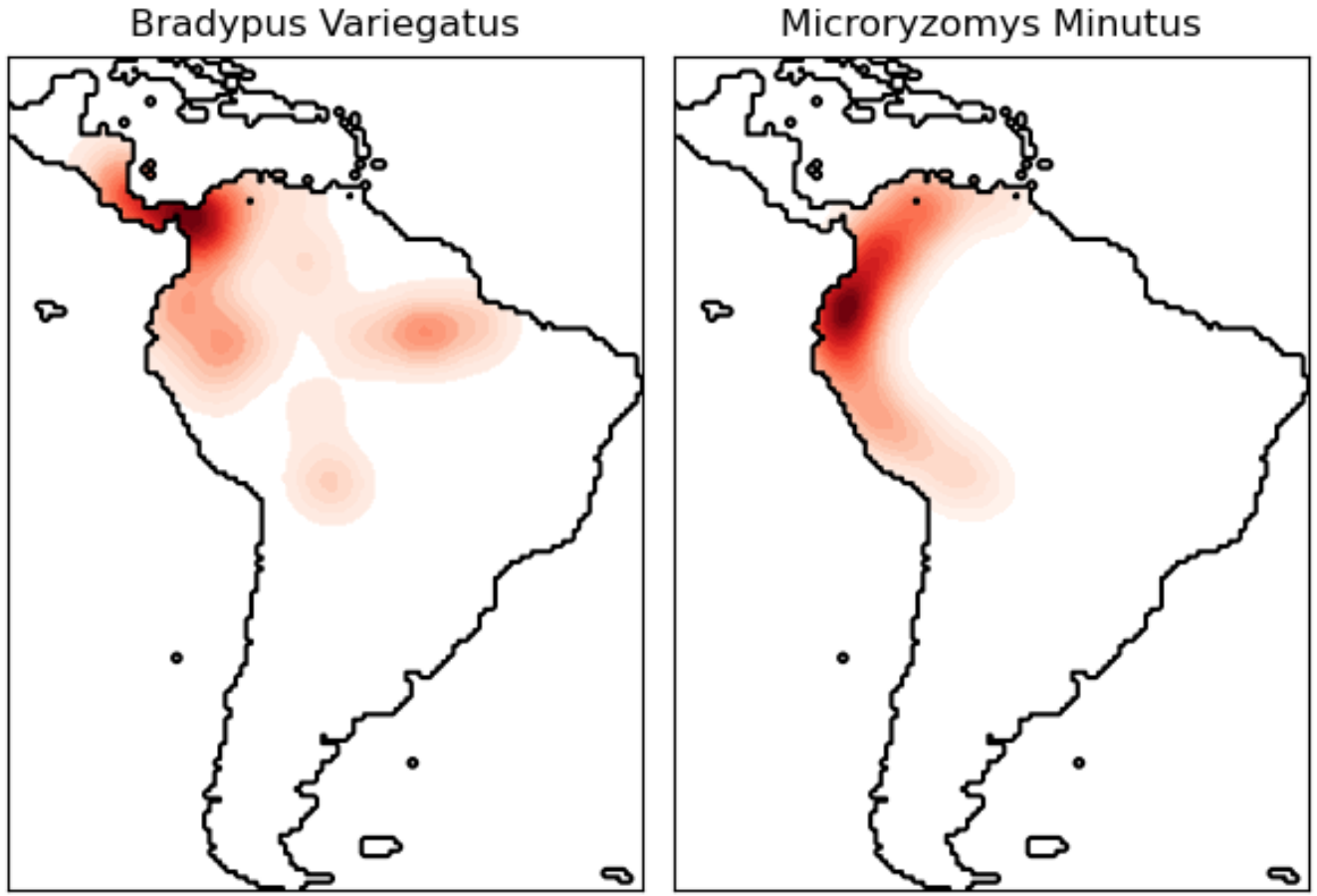
**Transfer:** The learned structure can be exploited for more than just generation (e.g. remember MAEs).

# Kernel Density Estimation




Phillips et al, Ecological Modelling (2006)

# Kernel Density Estimation

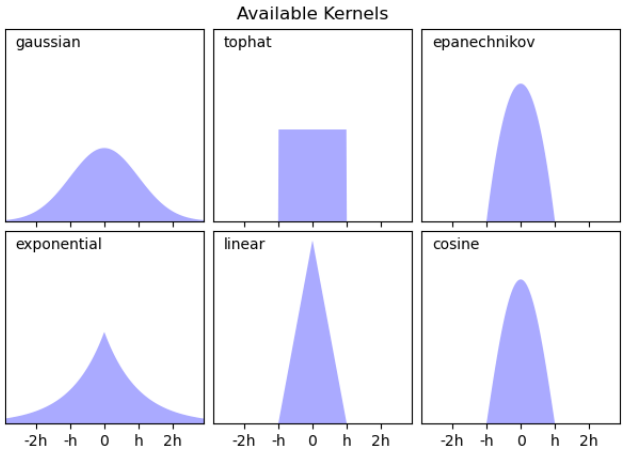


# Kernel Density Estimation

Lays a „kernel“ around every data point and sums them up
-> each data point contributes probability density in its proximity

$$\hat{f}_h(x) = \frac{1}{n}\sum_{i=1}^{n} K_h(x - x_i) = \frac{1}{nh}\sum_{i=1}^{n} K\left(\frac{x - x_i}{h}\right)$$

different Kernels in sklearn:



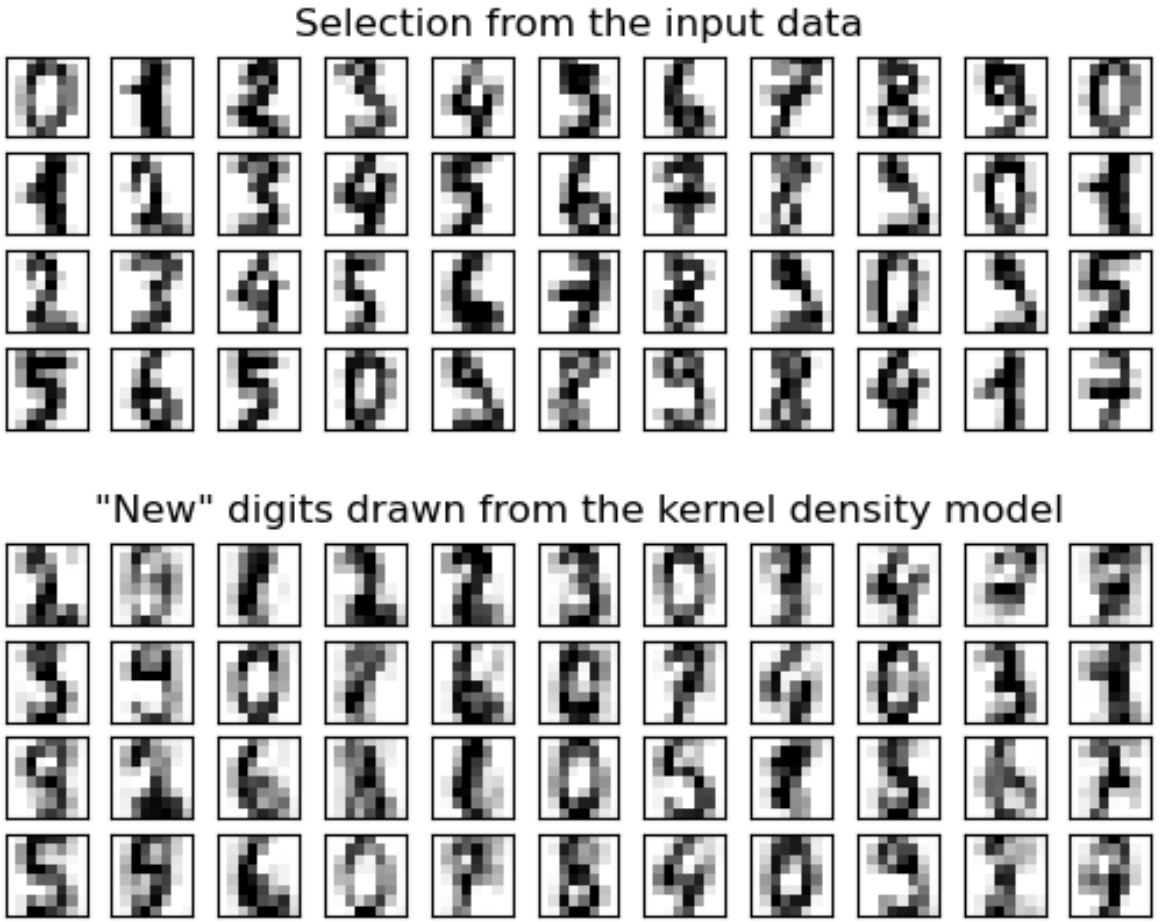
# Kernel Density Estimation

Lays a „kernel“ around every data point and sums them up
-> each data point contributes probability density in its proximity

$$\hat{f}_h(x) = \frac{1}{n}\sum_{i=1}^{n} K_h(x - x_i) = \frac{1}{nh}\sum_{i=1}^{n} K\left(\frac{x - x_i}{h}\right)$$

with Gaussian kernel: $$\hat{f}_h(x) = \frac{1}{n}\sum_{i=1}^{n} \frac{1}{h\sqrt{2\pi}} \exp\left(\frac{-(x - x_i)^2}{2h^2}\right)$$

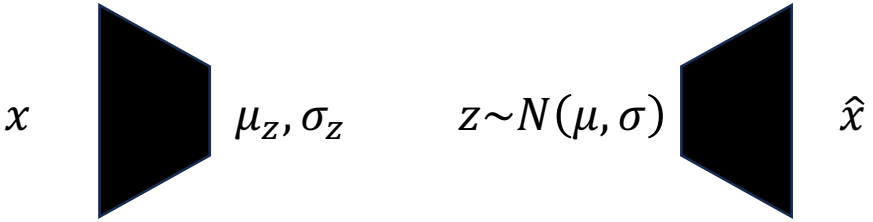
# Generative Modelling with KDE



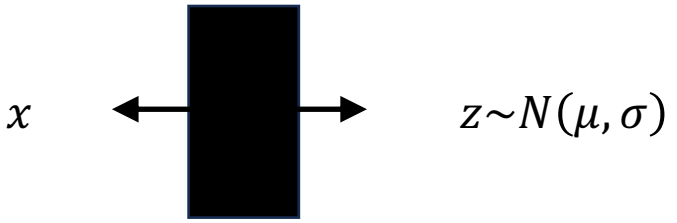

sklearn User Guide

# Normalizing Flows

Variational Autoencoder



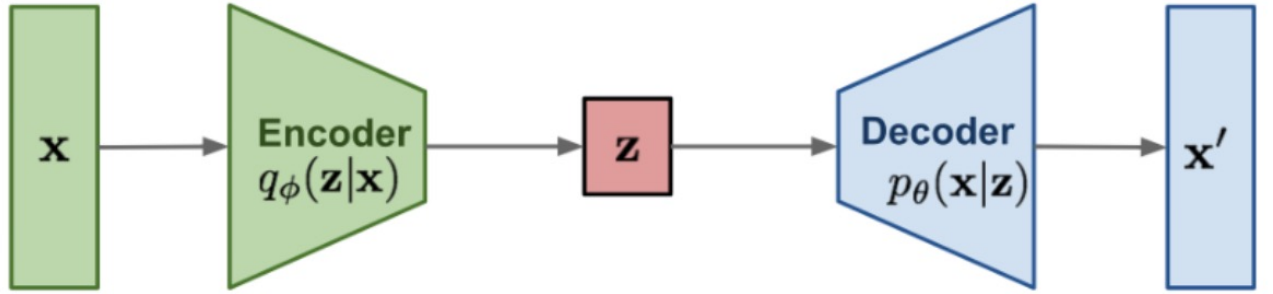

Normalizing Flows:



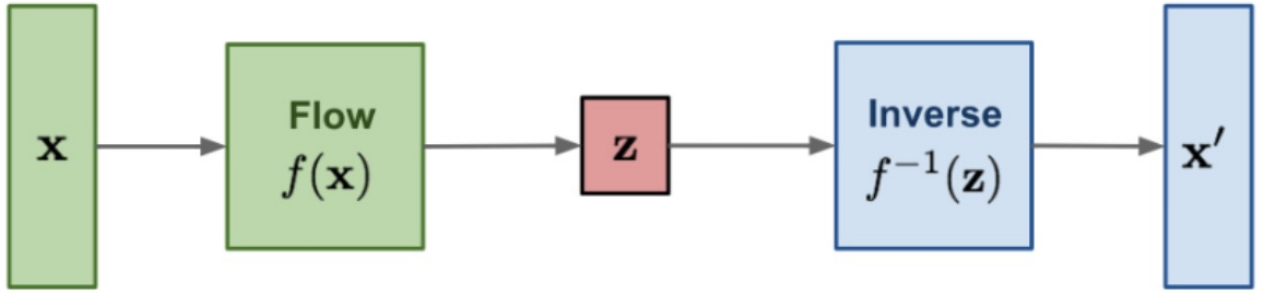

bijective network

# Normalizing Flows

Variational Autoencoder

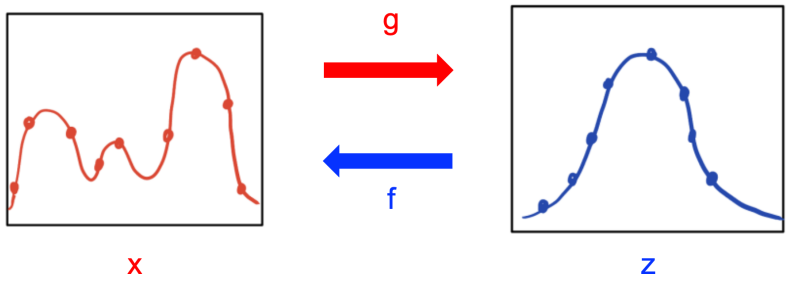



Normalizing Flows:




slide by Agarwal & Mulcahy, UW

# Normalizing Flows




$$\int p_x(x)dx = \int p_z(g(x))dz$$

$$p_x(x) = p_z(g(x))\left|\frac{dg(x)}{dx}\right| \rightarrow p_z(g(x))|det.J(g(x))|$$

$$\log p_x(x) = \log p_z(g(x)) + \log|det.J(g(x))|$$

**Intuitively**

z = g(x) determines where a point in x-space maps to z-space (where to move grains of sand)

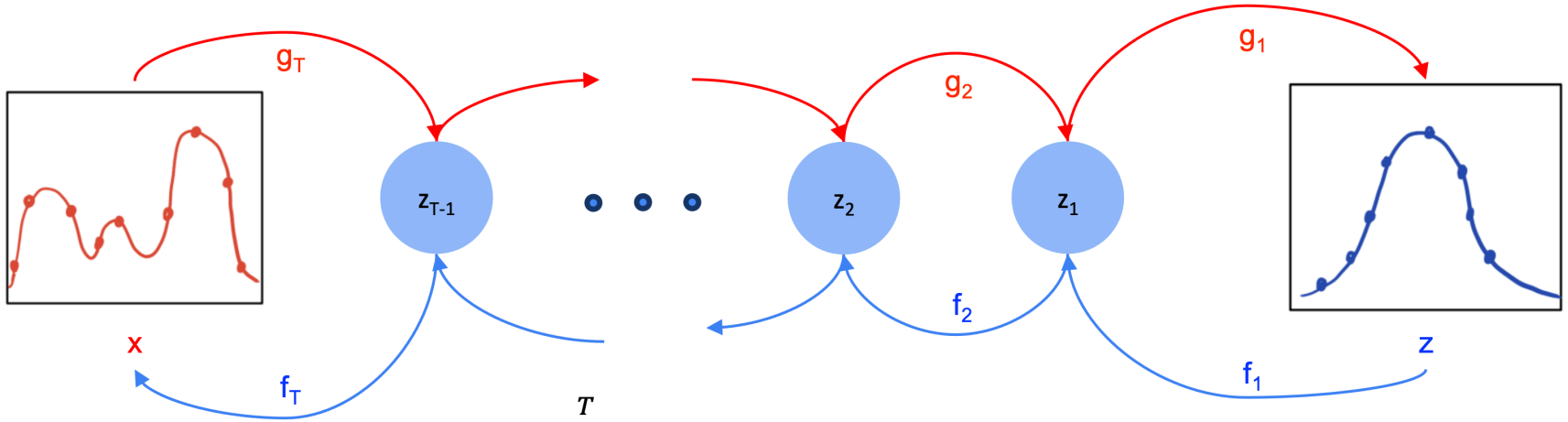
|det. J(g(x))| describes how much probability mass (sand) gets moved in a local neighborhood.

slide by Abuzar Khan, CMU

# Normalizing Flows

Composing many layers together:

$$z \sim p(z)$$
$$x = f_T \circ f_{T-1} \circ \cdots \circ f_1\,(z)$$




$$\log p_x(x) = \log p_z(z_0) + \sum_{t=1}^{T} \log\left|det.J_{z_t}\left(g(z_{t-1})\right)\right|, \qquad z_T = x,\ z_0 = z$$

**Inverse:** $z = g_1 \circ g_2 \circ \cdots \circ g_T(x)$

# How should the architecture of a NF look like?

- Simple prior pZ (z) that allows for efficient sampling and tractable likelihood evaluation. E.g., isotropic Gaussian
- Invertible transformations with tractable evaluation:
  - Likelihood evaluation requires efficient evaluation of $x \rightarrow z$ mapping
  - Sampling requires efficient evaluation of $z \rightarrow x$ mapping
- Computing likelihoods also requires the evaluation of determinants of the Jacobian matrices
  - **Key idea:** Choose tranformations so that the resulting Jacobian matrix has special structure. For example, the determinant of a triangular matrix is the product of the diagonal entries, i.e., an O(n) operation

slide by Stefano Ermon

# Real NVP

Published as a conference paper at ICLR 2017

## DENSITY ESTIMATION USING REAL NVP

**Laurent Dinh***
Montreal Institute for Learning Algorithms
University of Montreal
Montreal, QC H3T1J4

**Jascha Sohl-Dickstein**
Google Brain

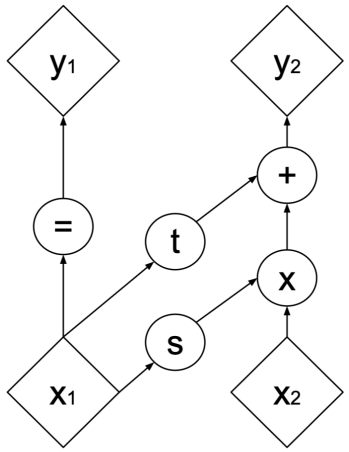
**Samy Bengio**
Google Brain

[cs.LG] 27 Feb 2017

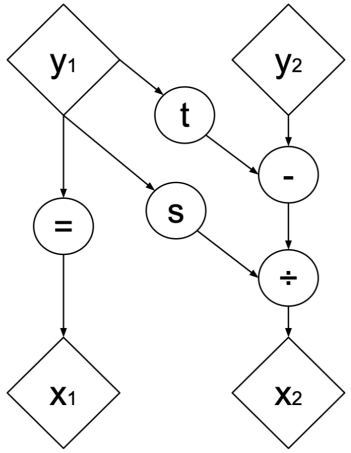
### ABSTRACT

Unsupervised learning of probabilistic models is a central yet challenging problem in machine learning. Specifically, designing models with tractable learning, sampling, inference and evaluation is crucial in solving this task. We extend the space of such models using real-valued non-volume preserving (real NVP) transformations, a set of powerful, stably invertible, and learnable transformations, resulting in an unsupervised learning algorithm with exact log-likelihood computation, exact and efficient sampling, exact and efficient inference of latent variables, and an

# Simple & Invertible: Affine Coupling Layers




(a) Forward propagation




(b) Inverse propagation

Transform is only applied to half of the dimensions:

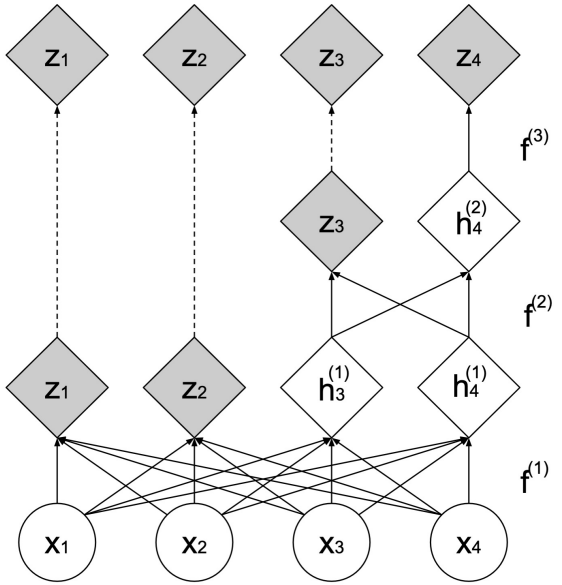
$$y_{1:d} = x_{1:d}$$
$$y_{d+1:D} = x_{d+1:D} \odot \exp\left(s(x_{1:d})\right) + t(x_{1:d}),$$

The Jacobian is a lower triangular, i.e. the determinant just factorizes the diagonal:

$$\frac{\partial y}{\partial x^T} = \begin{bmatrix} \mathbb{I}_d & 0 \\ \frac{\partial y_{d+1:D}}{\partial x_{1:d}^T} & \operatorname{diag}\left(\exp\left[s\left(x_{1:d}\right)\right]\right) \end{bmatrix}$$

# Reducing Dimensionality by Factorization

A common problems with NFs: z and x are of the same dimensionlity. This makes it expensive to model distributions of high-resolution images.











# GLOW

## Glow: Generative Flow with Invertible 1×1 Convolutions

**Diederik P. Kingma**[*†], **Prafulla Dhariwal**[*]
[*]OpenAI
[†]Google AI

### Abstract

Flow-based generative models (Dinh et al., 2014) are conceptually attractive due to tractability of the exact log-likelihood, tractability of exact latent-variable inference, and parallelizability of both training and synthesis. In this paper we propose *Glow*, a simple type of generative flow using an invertible $1 \times 1$ convolution. Using our method we demonstrate a significant improvement in log-likelihood on standard benchmarks. Perhaps most strikingly, we demonstrate that a flow-based generative model optimized towards the plain log-likelihood objective is capable of efficient realistic-looking synthesis and manipulation of large images. The code for our model is available at `https://github.com/openai/glow`.

### 1 Introduction

Two major unsolved problems in the field of machine learning are (1) data-efficiency: the ability to

# Key Innovation: Invertible 1x1 conv

a 1x1 conv is simply a matrix multiplication with kernel W along a single tensor axis.

The log-determinant of an invertible $1 \times 1$ convolution of a $h \times w \times c$ tensor $\mathbf{h}$ with $c \times c$ weight matrix $\mathbf{W}$ is straightforward to compute:

$$\log \left| \det \left( \frac{d\, \texttt{conv2D}(\mathbf{h}; \mathbf{W})}{d\, \mathbf{h}} \right) \right| = h \cdot w \cdot \log |\det(\mathbf{W})| \tag{9}$$

**LU Decomposition.** This cost of computing $\det(\mathbf{W})$ can be reduced from $\mathcal{O}(c^3)$ to $\mathcal{O}(c)$ by parameterizing $\mathbf{W}$ directly in its LU decomposition:

$$\mathbf{W} = \mathbf{PL}(\mathbf{U} + \text{diag}(\mathbf{s})) \tag{10}$$

$$\log |\det(\mathbf{W})| = \texttt{sum}(\log |\mathbf{s}|)$$

DANGER: W might collapse to a non-invertible (det(W) = 0) matrix during training. Needs to be monitored and training reset to last checkpoint.



Figure 4: Random samples from the model, with temperature 0.7.

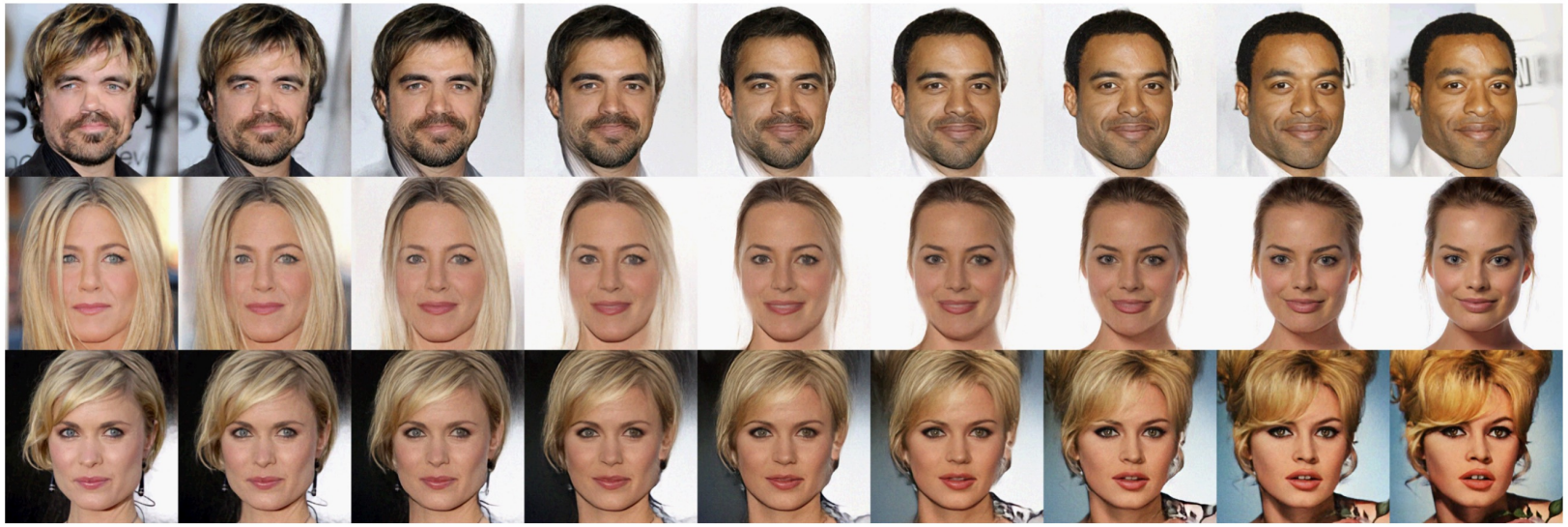


Figure 5: Linear interpolation in latent space between real images.

# Conditional Normalizing Flows

Preprint 2019

## LEARNING LIKELIHOODS WITH CONDITIONAL NORMALIZING FLOWS

**Christina Winkler, Daniel Worrall, Emiel Hoogeboom & Max Welling**
Institute of Informatics
University of Amsterdam
Netherlands
christina.winkler@student.uva.nl
{d.e.worrall, e.hoogeboom, m.welling}@uva.nl

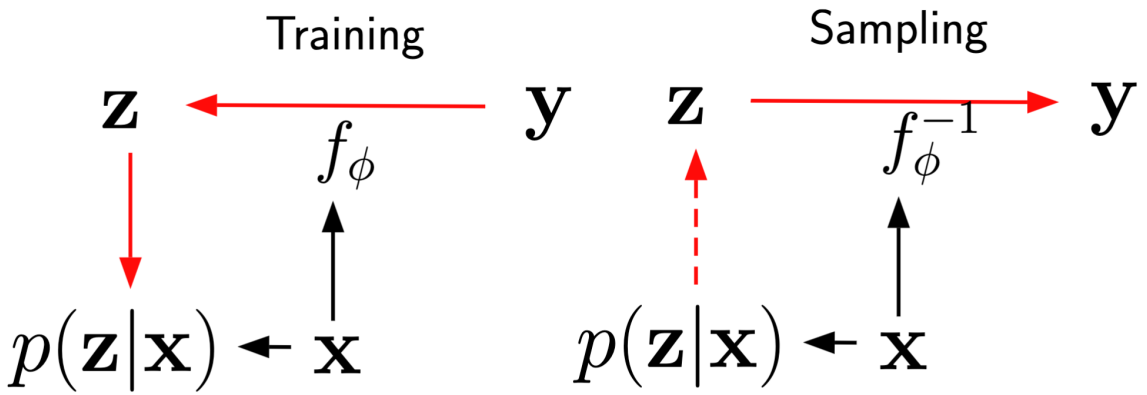
[cs.LG] 12 Nov 2023

### ABSTRACT

Normalizing Flows (NFs) are able to model complicated distributions $p_Y(y)$ with strong inter-dimensional correlations and high multimodality by transforming a simple base density $p_Z(z)$ through an invertible neural network under the change of variables formula. Such behavior is desirable in multivariate structured prediction tasks, where handcrafted per-pixel loss-based methods inadequately capture strong correlations between output dimensions. We present a study of conditional normalizing flows (CNFs), a class of NFs where the base density to output space mapping is conditioned on an input x, to model conditional densities

# Layers and Latent get conditioned on X

WARNING: Notation change. Here, the data is considered Y and the condition X.




| | |
|---|---|
| Conditional Prior | $p(\mathbf{z}\|\mathbf{x}) = \mathcal{N}(\mathbf{z}; \mu(\mathbf{x}), \sigma^2(\mathbf{x}))$ |
| Conditional Split Prior | $p(\mathbf{z}_1\|\mathbf{z}_0, \mathbf{x}) = \mathcal{N}(\mathbf{z}_1; \mu(\mathbf{z}_0, \mathbf{x}), \sigma^2(\mathbf{z}_0, \mathbf{x}))$ |
| Conditional Coupling | $\mathbf{y}_0 = s(\mathbf{z}_1, \mathbf{x}) \cdot \mathbf{z}_0 + t(\mathbf{z}_1, \mathbf{x}); \quad \mathbf{y}_1 = \mathbf{z}_1$ |

# Application: Super-Resolution





# Summary: Normalizing Flows

can be used for density estimation and data generation

simple idea

not very easy to train

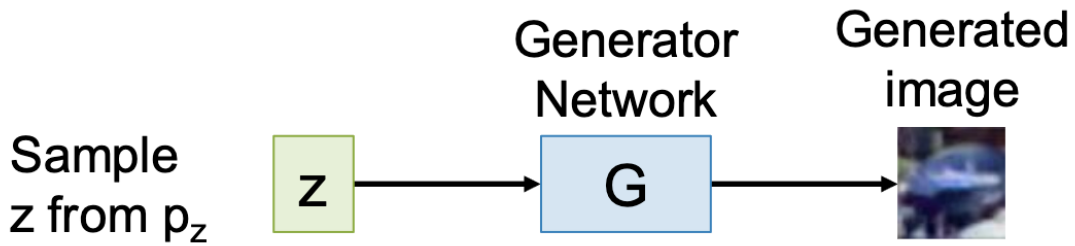
architecture restricted to bijective mappings and simple Jacobians

x and z have same dimensionality -> requires tricks to reduce computational load

# Generative Adversarial Networks

**Setup**: Have data $x_i$ drawn from distribution $p_{data}(x)$. Want to sample from $p_{data}$

**Idea**: Introduce a latent variable z with simple prior p(z) (e.g. unit Gaussian)
Sample $z \sim p(z)$ and pass to a **Generator Network** x = G(z)
Then x is a sample from the **Generator distribution** $p_G$. Want $p_G = p_{data}$!

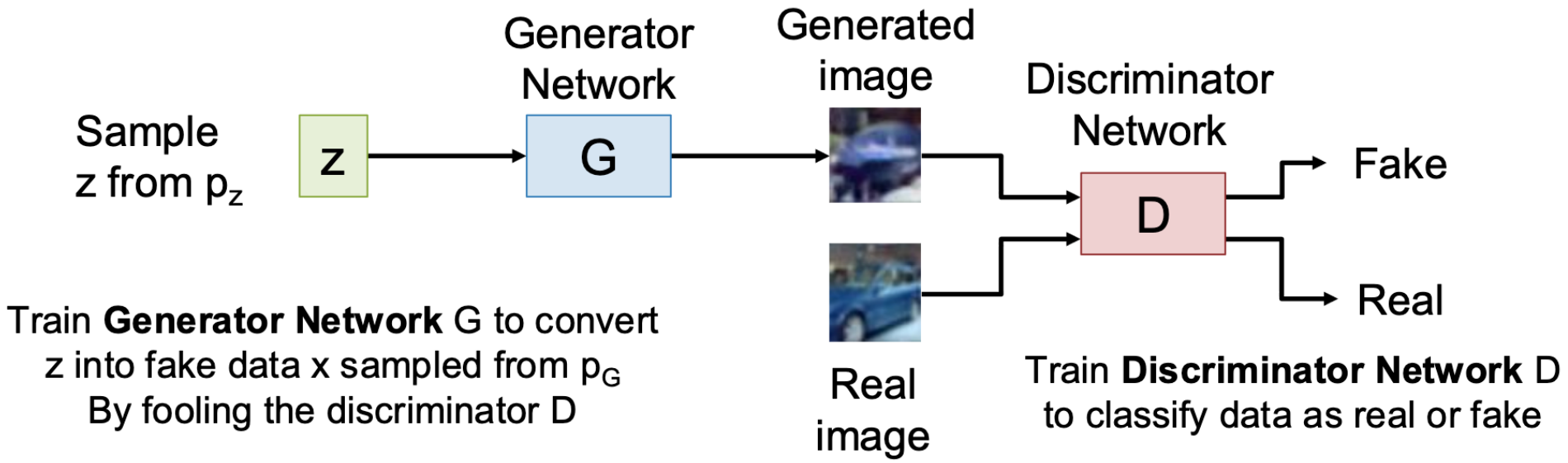



Train **Generator Network** G to convert z into fake data x sampled from $p_G$

# Generative Adversarial Networks

**Setup**: Have data $x_i$ drawn from distribution $p_{data}(x)$. Want to sample from $p_{data}$

**Idea**: Introduce a latent variable z with simple prior p(z) (e.g. unit Gaussian)
Sample $z \sim p(z)$ and pass to a **Generator Network** x = G(z)
Then x is a sample from the **Generator distribution** $p_G$. Want $p_G = p_{data}$!

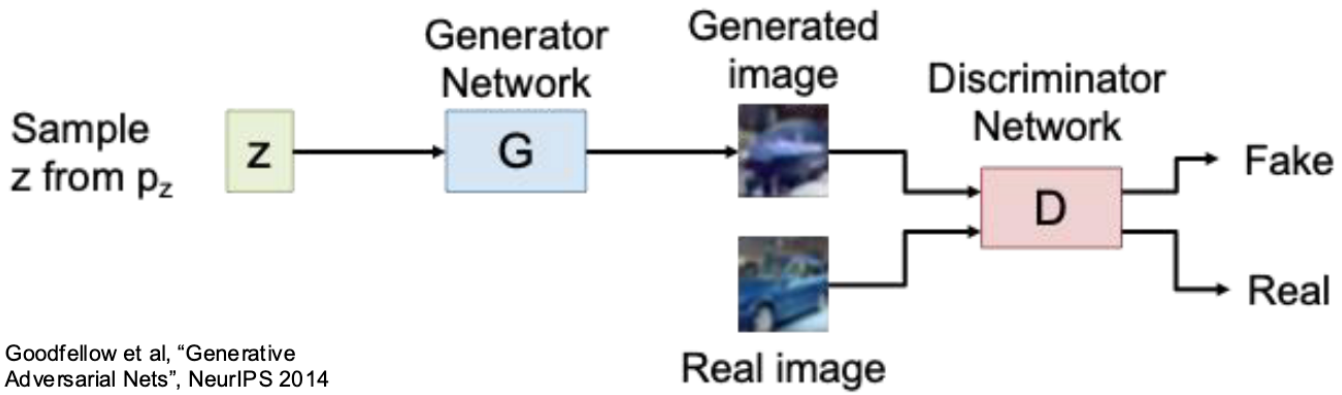



slide by Li et al, Stanford CS231n

# Generative Adversarial Networks

Jointly train generator G and discriminator D with a **minimax game**

$$\min_{G} \max_{D} \left( E_{x \sim p_{data}} [\log D(x)] + E_{z \sim p(z)} \left[ \log \left( 1 - D(G(z)) \right) \right] \right)$$




Goodfellow et al, "Generative Adversarial Nets", NeurIPS 2014

D(x) = P(x is real)
D(x) = 0 => fake
D(x) = 1 => real

slide by Li et al, Stanford CS231n

# Generative Adversarial Networks

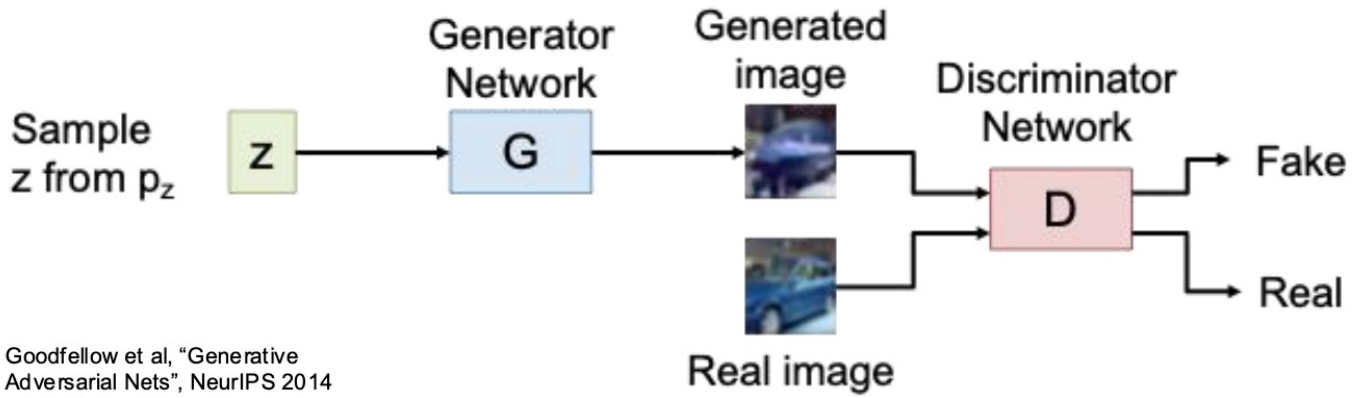
Jointly train generator G and discriminator D with a **minimax game**

Discriminator wants D(x) = 1 for real data

Discriminator wants D(x) = 0 for fake data

Imagine fixing G

$$\min_G \max_D \left( E_{x \sim p_{data}} [\log D(x)] + E_{z \sim p(z)} \left[ \log \left( 1 - D(G(z)) \right) \right] \right)$$




Goodfellow et al, "Generative Adversarial Nets", NeurIPS 2014

D(x) = P(x is real)
D(x) = 0 => fake
D(x) = 1 => real

slide by Li et al, Stanford CS231n

# Generative Adversarial Networks

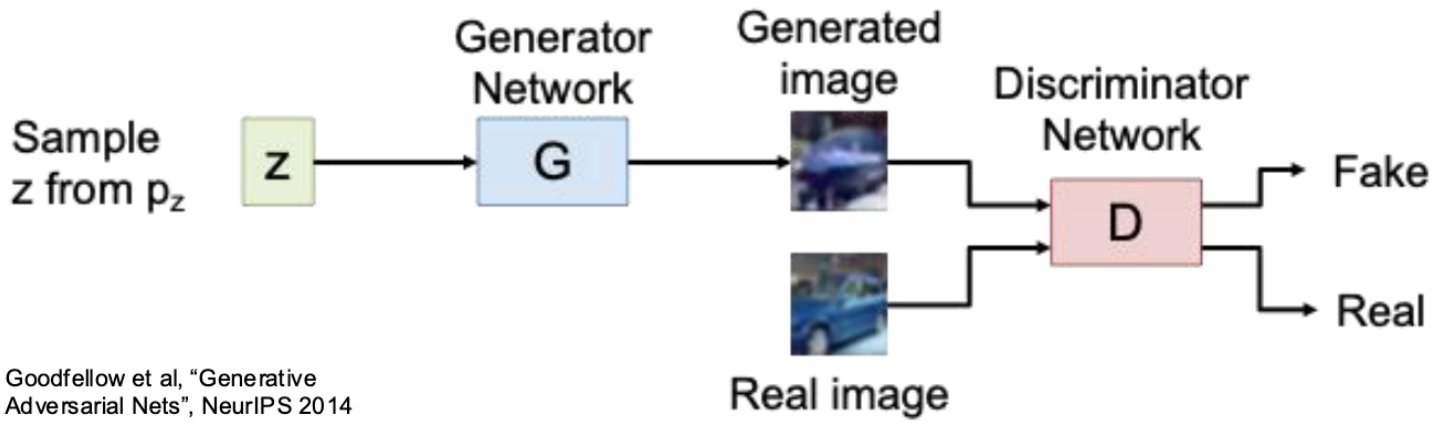
Jointly train generator G and discriminator D with a **minimax game**

This term does not depend on G

Generator wants D(x) = 1 for fake data

Imagine fixing **D**

$$\min_G \max_D \left( E_{x \sim p_{data}}[\log D(x)] + E_{z \sim p(z)}\left[\log\left(1 - D(G(z))\right)\right] \right)$$




Goodfellow et al, "Generative Adversarial Nets", NeurIPS 2014

D(x) = P(x is real)
D(x) = 0 => fake
D(x) = 1 => real

slide by Li et al, Stanford CS231n

# Style GAN

## A Style-Based Generator Architecture for Generative Adversarial Networks

Tero Karras
NVIDIA
tkarras@nvidia.com

Samuli Laine
NVIDIA
slaine@nvidia.com

Timo Aila
NVIDIA
taila@nvidia.com

arXiv:1812.04948v3 [cs.NE] 29 Mar 2019

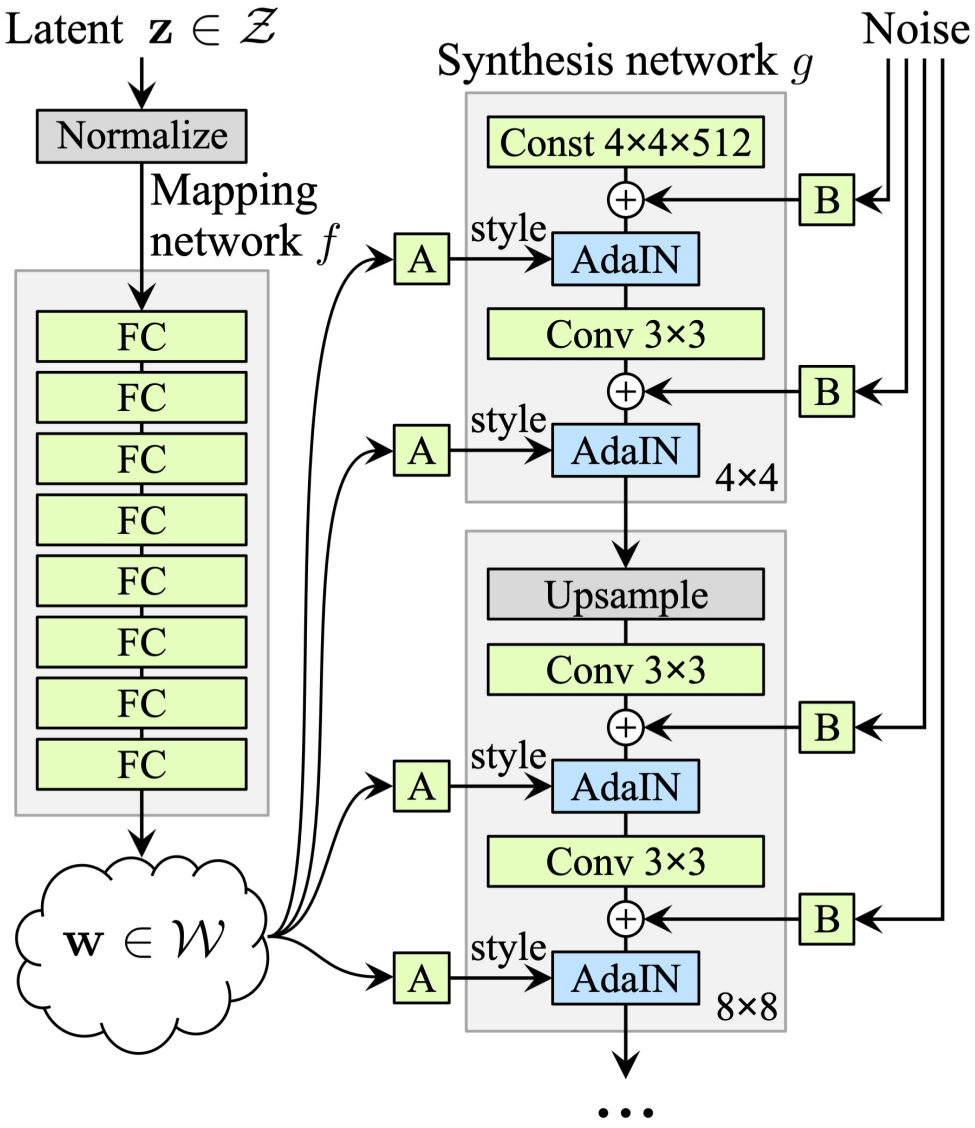
### Abstract

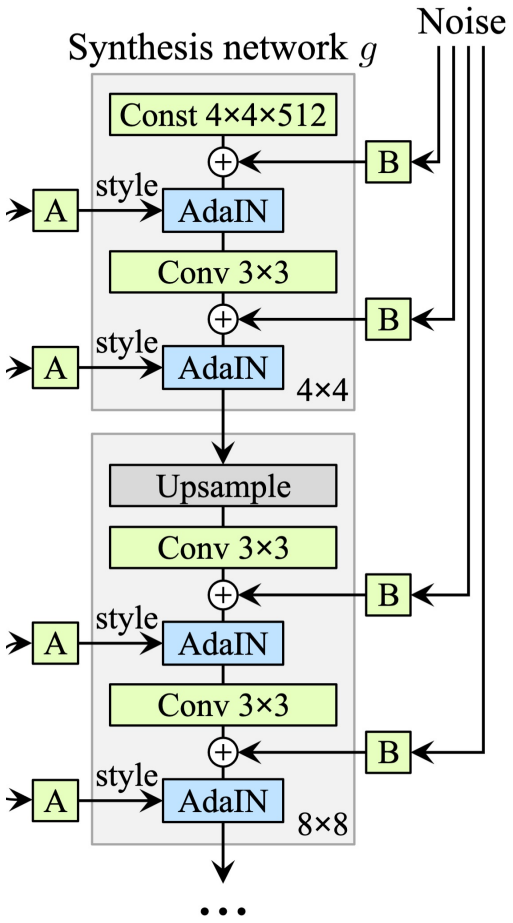
*We propose an alternative generator architecture for generative adversarial networks, borrowing from style transfer literature. The new architecture leads to an automatically learned, unsupervised separation of high-level attributes (e.g., pose and identity when trained on human faces) and stochastic variation in the generated images (e.g., freckles, hair), and it enables intuitive, scale-specific control of the synthesis. The new generator improves the state-of-the-art in terms of traditional distribution quality metrics, leads to demonstrably better interpolation properties, and also better disentangles the latent factors of variation. To quantify interpolation quality and disentanglement, we propose two new, automated methods that are applicable to any generator architecture. Finally, we introduce a*

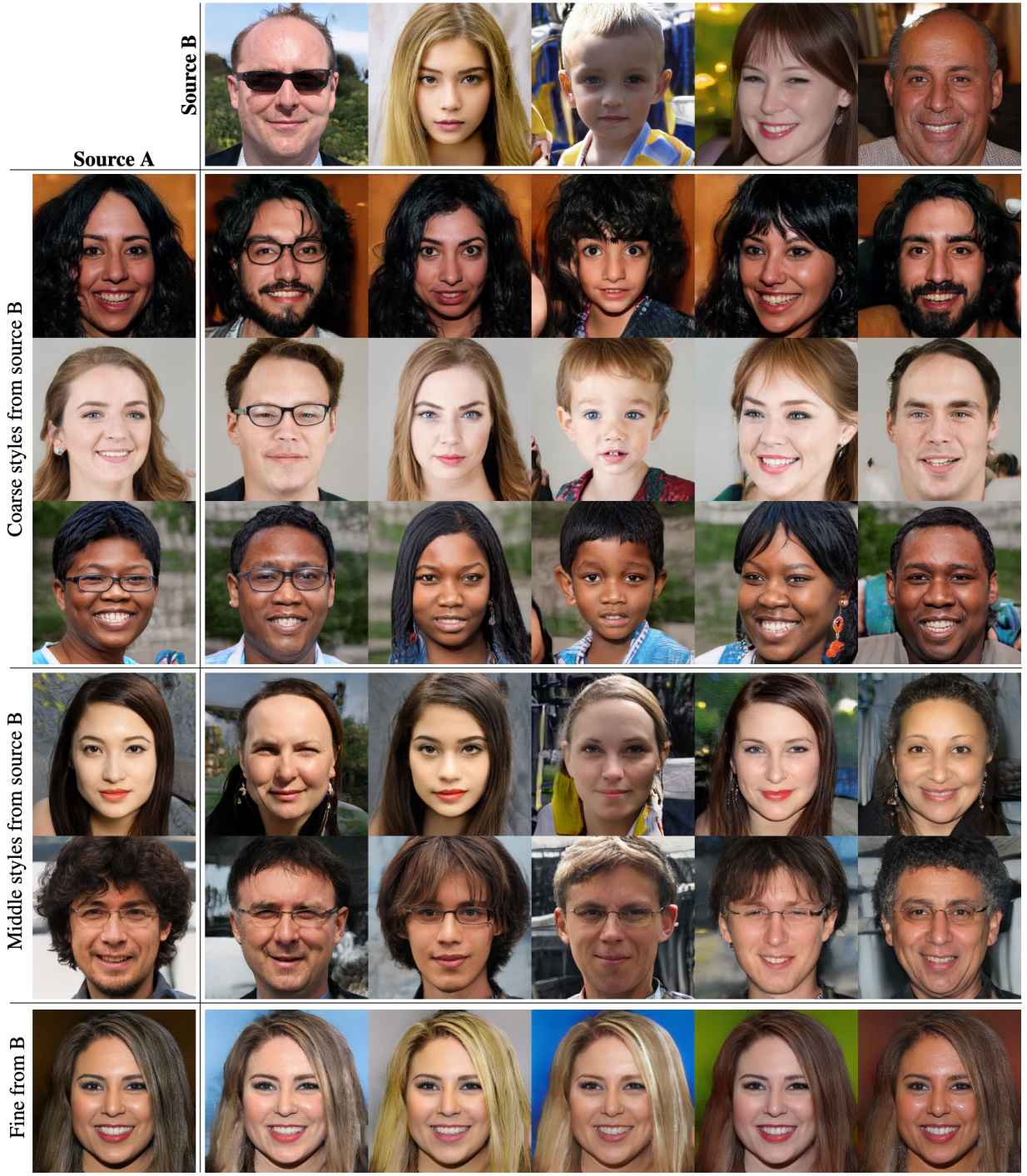
(e.g., pose, identity) from stochastic variation (e.g., freckles, hair) in the generated images, and enables intuitive scale-specific mixing and interpolation operations. We do not modify the discriminator or the loss function in any way, and our work is thus orthogonal to the ongoing discussion about GAN loss functions, regularization, and hyperparameters [24, 45, 5, 40, 44, 36].

Our generator embeds the input latent code into an intermediate latent space, which has a profound effect on how the factors of variation are represented in the network. The input latent space must follow the probability density of the training data, and we argue that this leads to some degree of unavoidable entanglement. Our intermediate latent space is free from that restriction and is therefore allowed to be disentangled. As previous methods for estimating the degree of latent space disentanglement are not directly appli-

# Style GAN


Latent $\mathbf{z} \in \mathcal{Z}$
Normalize
Mapping network $f$
FC
FC
FC
FC
FC
FC
FC
FC
$\mathbf{w} \in \mathcal{W}$
Synthesis network $g$
Noise
Const 4×4×512
B
style
A
AdaIN
Conv 3×3
B
style
A
AdaIN
4×4
Upsample
Conv 3×3
B
style
A
AdaIN
Conv 3×3
B
style
A
AdaIN
8×8
...


Synthesis network $g$
Noise
Const 4×4×512
B
style
A
AdaIN
Conv 3×3
B
style
A
AdaIN
4×4
Upsample
Conv 3×3
B
style
A
AdaIN
Conv 3×3
B
style
A
AdaIN
8×8


Source B
Source A
Coarse styles from source B
Middle styles from source B
Fine from B

# What is “Disentanglement“

For a distribution p(X) that we link to a distribution p(Z), we often hope that our model learns to map X and Z in a way that linear subspaces of Z (i.e. varying Z along a single dimension) have semantic meaning X.

Examples:

- facial expression vs hair color
- leaf type vs size of a tree




We want a „SIMS Character Creator“ for our data.

# Noise is useful in generative tasks for fine, „random“ details

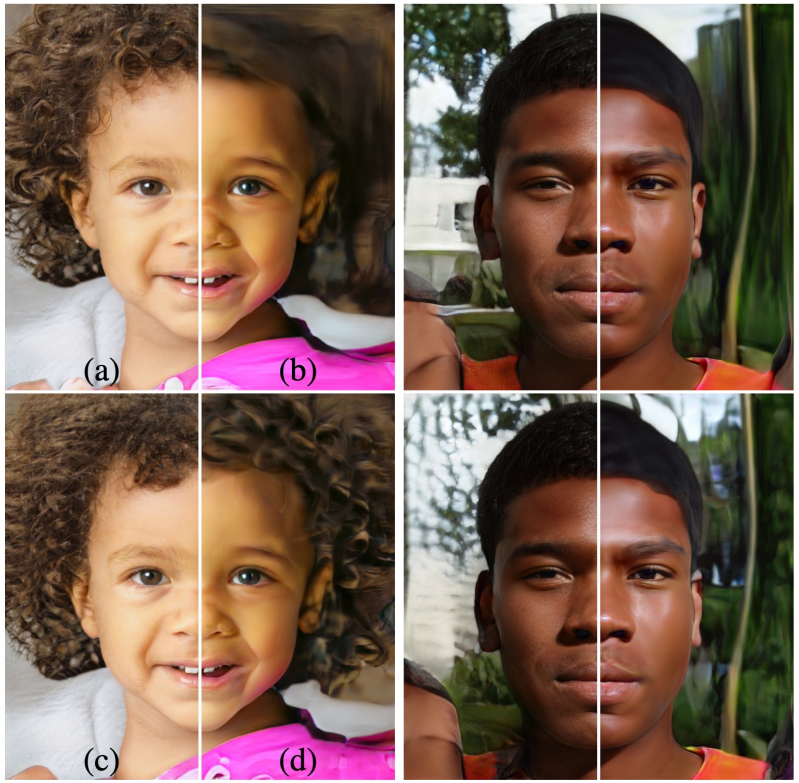



Figure 5. Effect of noise inputs at different layers of our generator. (a) Noise is applied to all layers. (b) No noise. (c) Noise in fine layers only ($64^2$ – $1024^2$). (d) Noise in coarse layers only ($4^2$ – $32^2$). We can see that the artificial omission of noise leads to featureless “painterly” look. Coarse noise causes large-scale curling of hair and appearance of larger background features, while the fine noise brings out the finer curls of hair, finer background detail, and skin pores.



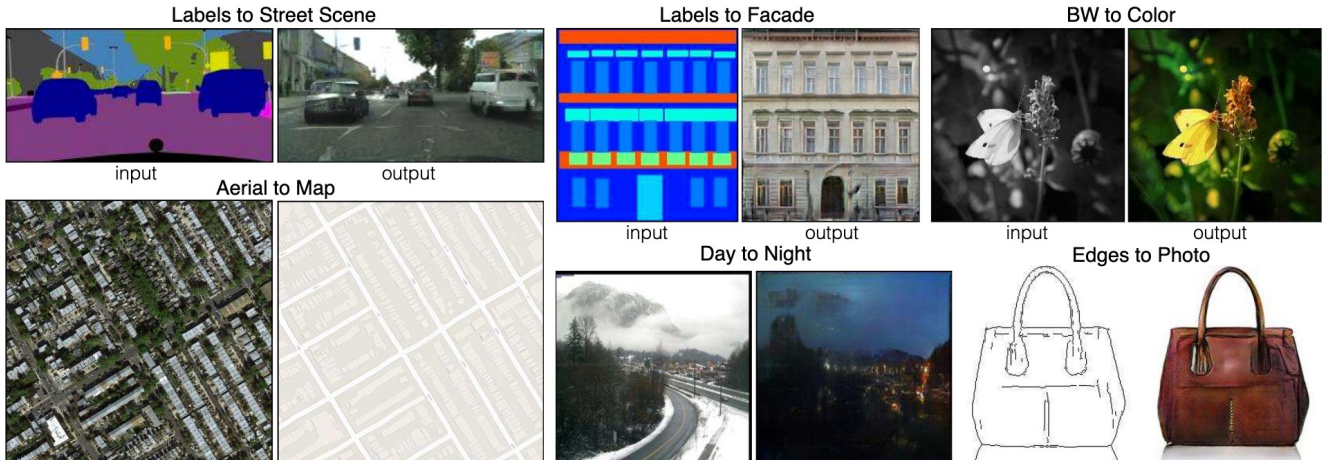
# Pix2Pix

This CVPR paper is the Open Access version, provided by the Computer Vision Foundation.
Except for this watermark, it is identical to the version available on IEEE Xplore.

## Image-to-Image Translation with Conditional Adversarial Networks

Phillip Isola Jun-Yan Zhu Tinghui Zhou Alexei A. Efros

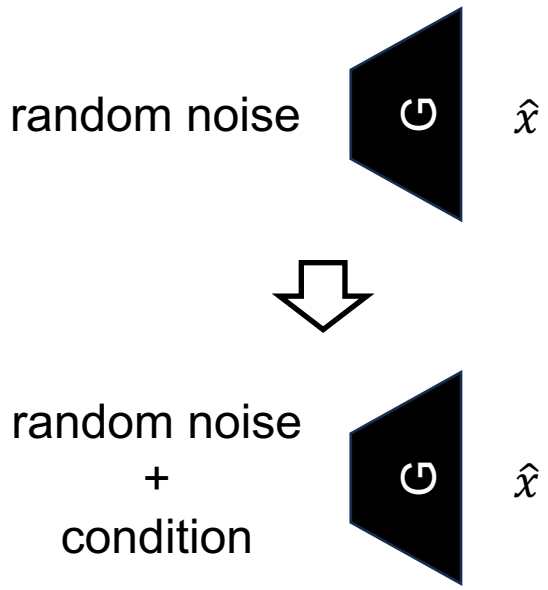
Berkeley AI Research (BAIR) Laboratory, UC Berkeley



# Conditional GAN

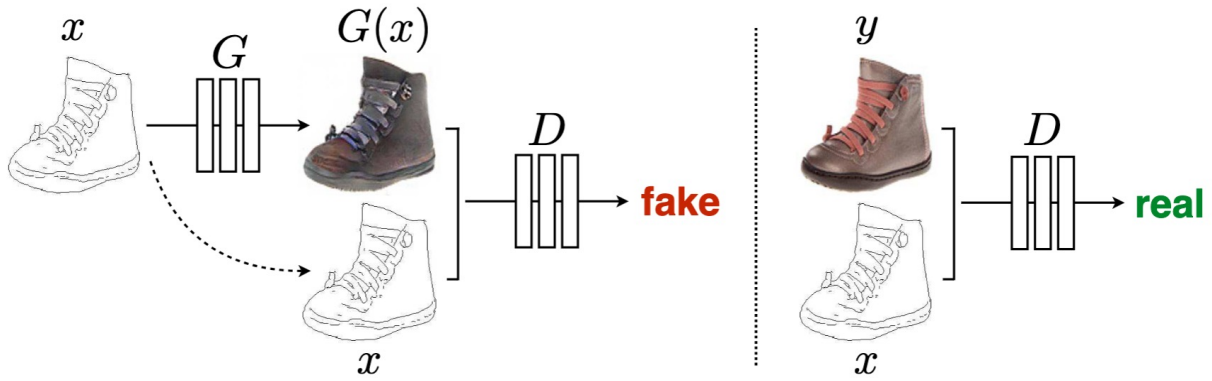
Like conditional NFs, the Generator of a GAN can take any input.

**Advantage** the GAN generator is not architecturally restricted



# Difference to classic GAN

The discriminator also sees the condition




The Generator gets an additional L1 Loss on the original image

$$G^* = \arg\min_G \max_D \mathcal{L}_{cGAN}(G, D) + \lambda \mathcal{L}_{L1}(G).$$

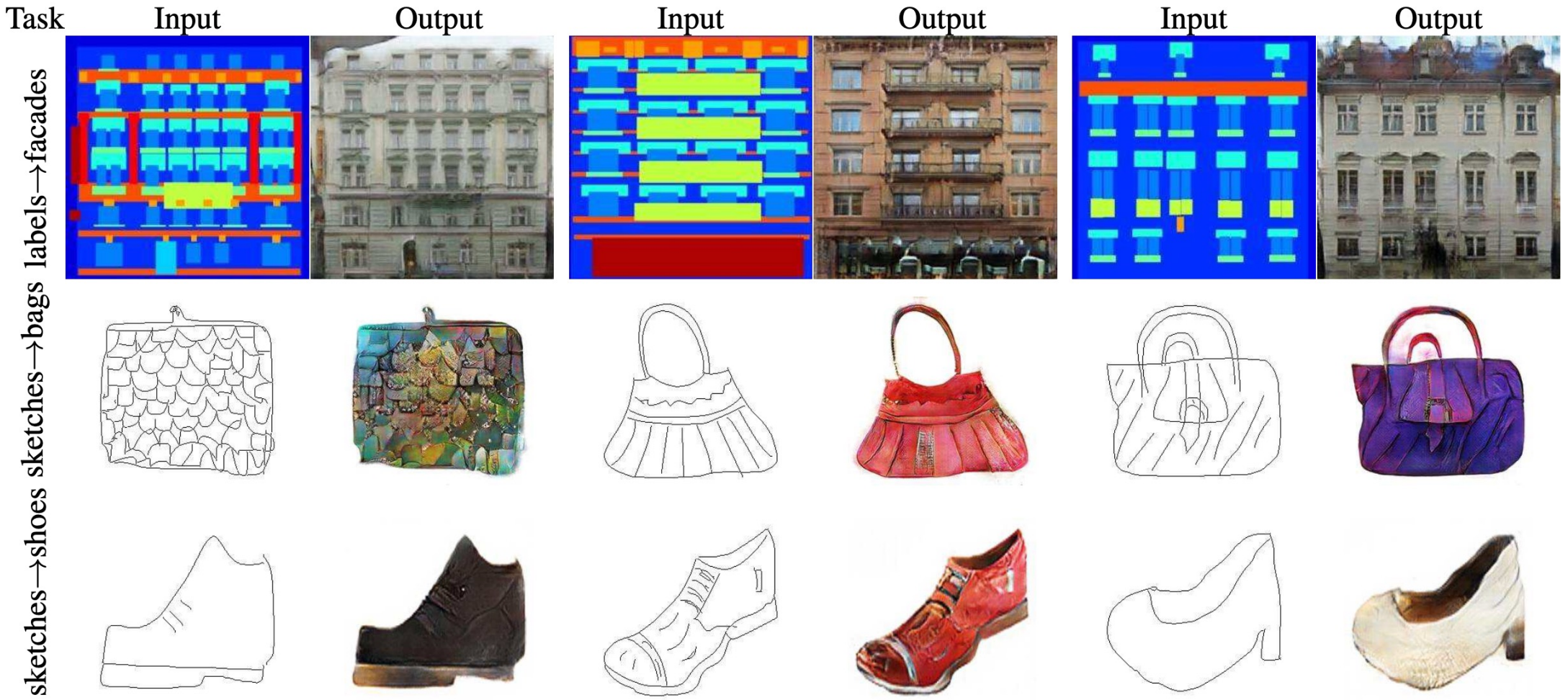



Figure 9: Results of our method on several tasks (data from [42] and [17]). Note that the sketch→photo results are generated by a model trained on automatic edge detections and tested on human-drawn sketches. Please see online materials for additional examples.

NOTE: Some of these tasks require labelled training data.

# Generative Adversarial Networks

Pros:
- Simple formulation
- Very good image quality

Cons:
- No loss curve to look at
- Unstable training
- Hard to scale to big models + data

slide by Li et al, Stanford CS231n

# Strengths of Generative Approach

**Modeling ambiguity:** If there are many possible outputs x for an input y, we want to model P(x | y)

**Language Modeling**: Produce output text x from input text y

*Write me a short rhyming poem about generative models*



*They sample from a learned **P**,*
*A distribution—structured, free.*
*Each token comes conditionally,*
*On all the ones that used to be.*

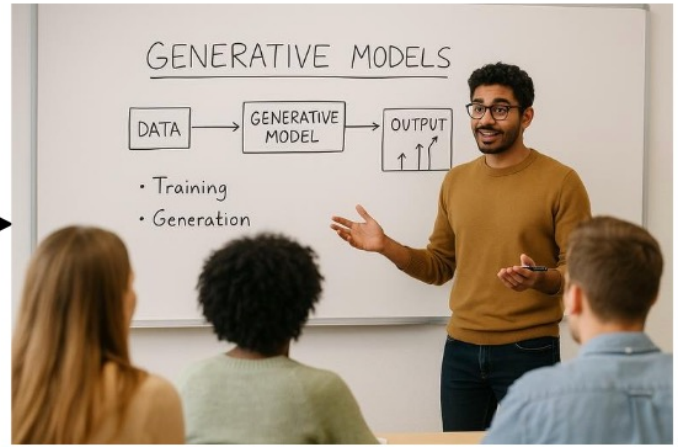
slide by Li et al, Stanford CS231n

# Strengths of Generative Approach

**Modeling ambiguity:** If there are many possible outputs x for an input y, we want to model P(x | y)

**Text to Image**: Produce output image x from input text y

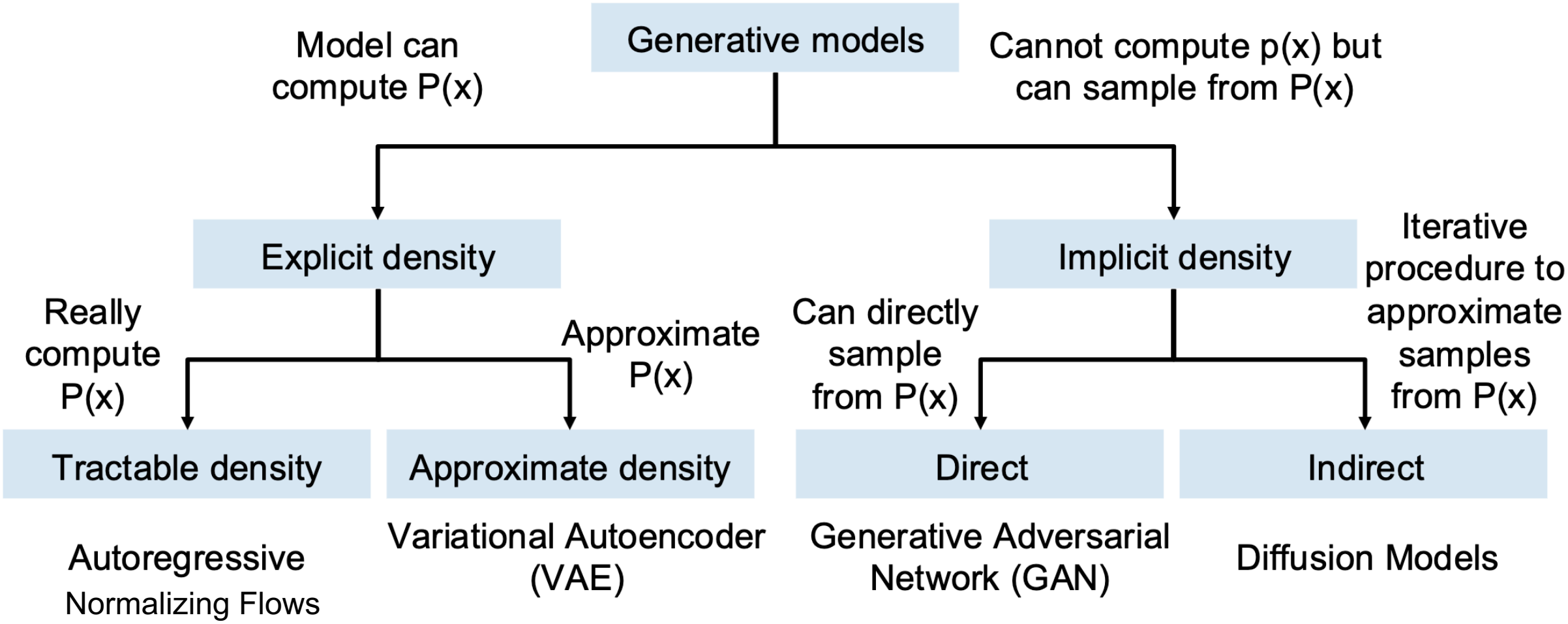
*Make me an image showing a person teaching a class on generative models in front of a whiteboard*




slide by Li et al, Stanford CS231n

# Taxonomy of Generative Models

Figure adapted from Ian Goodfellow, Tutorial on Generative Adversarial Networks, 2017




slide by Li et al, Stanford CS231n